

Week 3 - Completing the Decomposition

Let us complete the decomposition of an arbitrary unitary into the universal basis **{H, T, CNOT}**. Implement the functions below in order: each stage depends on the ones above it. Every item is currently an empty function or class that raises `NotImplementedError`; replace the body.

Conventions: a `Unitary` ($N \times N$) and a 2×2 gate block are both `numpy` complex arrays; a `ComplexVec` is a 1-D array. A `Circuit` is a list of gate objects, each with a `to_unitary()`; gates are stored in order of application (first gate applied first = rightmost matrix factor).

Stage 0 : `rotation.py` : single-qubit synthesis (done last week)

1. `to_bloch(g)` : recover the axis-angle (Bloch) form (α, n, θ) of a 2×2 unitary. Everything downstream that reasons about rotations starts here.
2. `n1n2n1_angles(b)` : factor a rotation into three angles about the fixed frame $R_{n_1}(\alpha) R_{n_2}(\beta) R_{n_1}(\gamma)$. This is how a gate becomes three angles we can approximate.
3. `approx_angle_with_tolerance(angle, tolerance)` : find an integer k with $k\lambda_\pi \approx \text{angle} \pmod{2\pi}$. Turns a continuous angle into a discrete number of H/T building-block applications.
4. `decompose_2x2(u, tolerance)` : combine 2 and 3 to get powers (k, l, m) with $u \approx M_1^k M_2^l M_1^m$. The core single-qubit $\rightarrow$ discrete-word step.

Stage 1 : `rotation.py` : rotation helpers (new)

5. `from_axis_angle(b)` : build a 2×2 unitary from its Bloch form (inverse of `to_bloch`). Needed to construct rotations from angles.
6. `Rz(theta)`, `Ry(theta)` : the textbook z/y rotations $R_z(\theta)$ and $R_y(\theta)$. The alphabet used by the ABC / Euler decompositions.
7. `euler_angles_zyz(u)` : ZYZ Euler angles $(\alpha, \beta, \gamma, \delta)$ with $u = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$. The algebraic heart of `abc_decompose`.
8. `unitary2_sqrt(u)` : a 2×2 matrix V with $VV = u$ (halve the Bloch α and θ). `decompose_controlled` needs $V = \sqrt{U}$ to build its recursion.
9. `expand_word`, `gates_to_unitary`, `invert_gates`, `power_gates` : In Week 2, I listed the expressions of M_1 , M_2 in terms of **sequences of powers of H, T (called words)**. H/T word utilities: expand exponent lists to flat 'H'/T' strings, evaluate a word to a matrix, invert a word (with $T^{-1} = T^7$), and raise a word to a power.
10. `approximate_in_ht(u, error)` : assemble the flat H/T word for u from `decompose_2x2`'s powers (k, l, m) and `power_gates`. The bridge into `decompose_to_ht`.

Stage 2 : `decompose.py` : gate representations

11. `num_qubits(N)` : the n with $2^n = N$. Used throughout to size registers.
12. `TwoLevel.to_unitary` : expand a two-level gate to the full $N \times N$ matrix.
13. `SingleQubitGate.to_unitary` : expand a 2×2 acting on one qubit to $N \times N$.
14. `ControlledU.to_unitary` : $C^k(U)$ controlled on all other qubits.
15. `CU.to_unitary` : singly-controlled $C(U)$ (keeps the global phase).
16. `CNOT.to_unitary` : controlled- X .
17. `circuit_to_unitary`, `to_circuit`, `error_up_to_phase` : evaluate a circuit to a matrix, wrap two-levels as a circuit, and compare two unitaries ignoring an overall global phase (the correctness metric for the whole pipeline).

Stage 3 : `decompose.py` : two-level decomposition (stage 1 of the pipeline)

18. `align(x, y, norm)` : the 2×2 rotation that zeroes one entry of a column. The atomic step of Gaussian-style elimination on a unitary.
19. `decompose_vector(vec)` : two-levels that turn a column into $(1, 0, 0, \dots)$.
20. `expand_twolevels`, `two_levels_to_unitary`, `adjoint_twolevel`, `adjoint_twolevels` : lift a sub-block decomposition to full size, evaluate a two-level sequence, and take adjoints (needed to invert the reduction).
21. `decompose_unitary(u)` : repeat `decompose_vector` over sub-columns to reduce u to identity; returns the sequence S with $\prod S = u^\dagger$.
22. `twolevel_decomposition(u)` : the two-level decomposition of u itself (the adjoint of `decompose_unitary`). **Stage 1 output.**

Stage 4 : `decompose.py` : ABC, gray code, controlled circuits

23. `abc_decompose(u)` : $U = e^{i\alpha} AXBXC$ with $ABC = I$ (Corollary 4.2); lets a controlled gate be built from CNOTs and single-qubit gates.
24. `abc_reconstruct(d)` : reassemble $e^{i\alpha} AXBXC$ (for checking).
25. `gray_code(tl)` : flips walking from `level0` to `level1`, so a two-level on far-apart levels becomes one on adjacent levels.
26. `decompose_swap`, `controlled_circuit` : realize a multi-controlled NOT / an arbitrarily-controlled 2×2 as a fully-controlled core sandwiched by X gates.

Stage 5 : `decompose.py` : the pipeline

27. `decompose_twolevel(tl)` : `TwoLevel` $\rightarrow$ `SingleQubitGate` + `ControlledU` via the gray-code walk. **Stage 2.**
28. `decompose_controlled(n, controls, target, u)` : $C^k(U) \rightarrow C(U)$ + CNOTs, recursively using $V = \sqrt{U}$ (fig 4.8).
29. `decompose_controlledU(g)` : `ControlledU` $\rightarrow$ CNOTs + $C(U)$ (all other qubits as controls). **Stage 3.**
30. `decompose_cu(g)` : `CU` $\rightarrow$ single-qubit gates + 2 CNOTs via ABC. **Stage 4.**

31. `decompose_to_basis(u)` : run stages 1–4 in sequence: Unitary $\rightarrow$ a circuit of only SingleQubitGate + CNOT.
32. `ht_gates, decompose_to_ht(u, error)` : replace each SingleQubitGate by its H/T word (via `rotation.approximate_in_ht`): Unitary $\rightarrow$ a circuit of only **H, T, CNOT**, matching u up to global phase. **Final result.**

Checking your work

Once the pipeline is done, verify end-to-end (e.g. in `main.py`): take a random unitary u , run `decompose_to_basis(u)` or `decompose_to_ht(u, error)`, rebuild with `circuit_to_unitary(...)`, and confirm `error_up_to_phase(u, rebuilt)` is near 0.

Task: check the scaling of `number_of_gates` vs error for a 4x4 unitary, and explain why it grows so rapidly.